LLM4Rec Homework Report — Assignment A03

Student: Anna Grishkina

Team: Individual

Date: 2026-09-30

Assignment: A03 — Collaborative Filtering (Week 3)

Live app: https://af851574-oss.github.io/RecSys-LLMs/week3/

Abstract

I implemented user-based and item-based collaborative filtering on the MovieLens 100K rating matrix (943 users × 1682 movies, 93.70% empty) in the instructor's Week 3 starter. The first version compared only co-rated entries, as the starter suggests. Testing showed that this made both Top-5 lists depend on almost no evidence: one shared movie always gives cosine 1.0, and the item-based list consisted of movies rated by one person in the whole dataset. In the second iteration I switched to similarity weighted by the number of co-rated items and required every recommendation to rest on at least five different users. On a fixed 10% hold-out, the user-based RMSE dropped from 1.0973 to 1.0405 and its coverage rose from 29.0% to 93.0%. A small matrix factorization model was still the most accurate (RMSE 0.9012).

Index Terms — collaborative filtering, cosine similarity, sparsity, cold start, matrix factorization, MovieLens 100K

1. Introduction

Problem statement. Given a user from `u.data`, recommend unseen movies twice — by similar users and by similar movies — and predict the rating of any chosen (user, movie) pair, as in the assignment mock-up. The starter [1] contained only the data loading and TODO stubs.

Motivation. Collaborative filtering uses other people's ratings instead of movie features [2], so it can recommend across genres, but it depends entirely on how much overlap exists in a very sparse matrix.

Concrete example. Users 1 and 166 share only two rated movies, yet their co-rated cosine is 0.902134. With one shared movie it would be exactly 1.0.

Contributions.

- Implemented the rating matrix, cosine similarity, user-based and item-based CF, Top-5 lists, and a pair-prediction panel in plain JavaScript.
- Measured all three missing-value strategies from the readme and a matrix factorization baseline on the same hold-out.
- Found and fixed two evidence problems in my first version, and documented the before/after results.

2. Related Work

The starter code and specification come from the course repository [1]. The Google course explains collaborative filtering through the feedback matrix and embeddings [2] and motivates matrix factorization [3]. The data and file schema come from MovieLens 100K [4], [5]. Herlocker et al. introduced weighting a neighbour's similarity by the number of co-rated items, with a cut-off at 50 [6]; my second iteration uses exactly this rule. Sarwar et al. showed that item-based CF can be computed offline and scales better when users outnumber items [7]. I considered Pearson correlation, but kept cosine because the readme specifies it and because the evidence problem is independent of the correlation formula.

3. Method

Representation. `ratingMatrix[user][movie]` holds ratings 1–5 and 0 for "not rated". Users are rows, which follows the starter code; the slide describes users as columns, but this is only a transpose.

Similarity. Raw cosine uses only co-rated entries: $\cos(a, b) = \Sigma a_i b_i / \sqrt{(\Sigma a_i^2 \cdot \Sigma b_i^2)}$ over indices where both are non-zero. The app uses the weighted strategy: $\text{sim} = \cos \cdot \min(n, 50)/50$, where n is the number of co-rated entries [6].

User-based CF. Take the 20 most similar users with positive similarity; predict $\hat{r}(u, j) = \Sigma \text{sim}(u, v) \cdot r(v, j) / \Sigma \text{sim}(u, v)$ over neighbours who rated j.

Item-based CF. Predict $\hat{r}(u, j) = \Sigma \text{sim}(i, j) \cdot r(u, i) / \Sigma \text{sim}(i, j)$ over the user's rated movies i, using positive item-item similarities. Both methods therefore output ratings on the same 1–5 scale.

Confidence filter. A candidate is shown only if at least `MIN_SUPPORT = 5` distinct other users stand behind the estimate. The pair prediction excludes the user's own rating of that movie (leave-one-out).

Tools and libraries.

- HTML, CSS, and browser JavaScript (no framework, no build step)
- Node.js v24.10.0 for `week3/test.mjs` and `week3/analysis.mjs` (`node:vm` over the real parser)
- Python 3 for serving and for my independent recomputation
- Google Chrome 152.0.7977.83 headless for the DOM check

- GitHub Pages for hosting

Configuration. N = 20 neighbours, shrinkage 50, MIN_SUPPORT = 5, Top-5. Evaluation: random 10% hold-out with seed 42, 2000 test pairs, fallback to the user mean when a method cannot predict. Matrix factorization: k = 20, 20 epochs, learning rate 0.01, regularization 0.05, seed 42.

4. Experiments

Setup. I first confirmed that the matrix has shape 944 × 1683 (index 0 unused) with exactly 100000 non-zero entries. Then I ran all methods through the same evaluation script twice to confirm deterministic output.

Method	RMSE	MAE	Coverage	Time
Global mean	1.1055	0.9264	100.0%	<1 s
User mean	1.0339	0.8343	100.0%	<1 s
User-based, co-rated only	1.0973	0.8614	29.0%	2.1 s
User-based, mean imputation	1.0698	0.8266	92.3%	3.8 s
User-based, weighted (app)	**1.0405**	**0.8090**	**93.0%**	1.9 s
Item-based, co-rated only	1.0335	0.8333	99.8%	1.0 s
Item-based, mean imputation	0.9686	0.7769	99.5%	1.9 s
Item-based, weighted (app)	**1.0281**	**0.8232**	**99.8%**	0.9 s
Matrix factorization	0.9012	0.7152	100.0%	0.18 s train

Coverage is the share of test pairs a method predicted by itself. Times are for all 2000 pairs.

Hand-computed example. Users 1 and 166 co-rated movie 243 (ratings 1 and 3) and movie 258 (5 and 4). Dot product = 1·3 + 5·4 = 23; norms = √(1 + 25) = √26 and √(9 + 16) = 5. Cosine = 23 / (5√26) = 0.902134; weighted = 0.902134 · 2/50 = 0.036085. Both values are asserted in the test.

Before and after for user 1. Numbers in brackets are the movie's total raters in the dataset.

Iteration 1 (co-rated only)	Iteration 2 (weighted, 5 users)
User-based: Money Talks 4.667 [45], The Postman 4.666 [58], The Wedding Singer 4.502 [72]	User-based: A Close Shave 4.857 [112], Hamlet 4.857 [90], Rear Window 4.801 [209]
Item-based: Cyclo 4.378 [1], Entertaining Angels 4.375 [1], Office Killer 4.279 [1]	Item-based: A Chef in Love 4.090 [8], Ruby in Paradise 4.037 [23], Wings of Desire 4.032 [57]

Pair prediction. User 1 rated Star Wars (1977) 5. Without that rating, user-based CF predicts 4.723 (18 users) and item-based CF predicts 3.698 (582 users). For user 100, who has not rated it, the predictions are 4.201 and 3.250.

Verification performed.

1. `node week3/test.mjs` — PASS: matrix shape and count, cosine identities, both hand-computed values, a tiny synthetic prediction (2.497205), valid Top-5 lists with support ≥ 5, leave-one-out, all TODOs removed.
2. `node week3/analysis.mjs` — the table above; two runs gave identical results apart from timings.
3. `node --check` on `script.js` and `data.js` — both exit 0; local `GET /week3/` — HTTP 200.
4. Headless Chrome showed the loaded page with 943 user options. It cannot click the button, so the recommendation paths were checked through the same core functions in Node.
5. I recomputed both Top-5 lists for users 1 and 100 and the three pair predictions in a separate Python script; every value matched.
6. `u.data`, `u.item`, and the starter `readme.md` are byte-identical to the instructor's version.

5. Discussion

Failure case. In iteration 1, 9 of 942 users had similarity exactly 1.0 to user 1, and 13 of the top-20 neighbours shared only one or two movies. The confidence filter counted the user's own movies, so an item-based candidate with a single rater still passed with support 37.

Root cause. For positive ratings, a cosine over one shared entry is x·y / (|x|·|y|) = 1. With 93.70% of cells empty, many pairs have that little overlap: across 10000 random user pairs, the median number of co-rated movies was 8, and 32.1% of pairs shared fewer than five.

User-based vs. item-based. With 943 users and 1682 movies, user-based CF compares against fewer entities and took about 2 ms per user with the app's core functions in Node; item-based CF compares every candidate with every rated movie and took about 1 s. Item-based CF was slightly more accurate (RMSE 1.0281 vs 1.0405) and covered almost every test pair. Its item-item similarities change slowly and can be precomputed offline, which is why it scales better when users greatly outnumber items [7].

Missing-value trade-off. Co-rated only is the simplest, but it trusts tiny overlaps. Mean imputation gave the best CF accuracy for items (0.9686), but it doubled the running time and treats "not seen" as "average", which biases the similarities. Weighting costs almost nothing and fixed the evidence problem. Matrix factorization was the most accurate and fast after training, but it requires hyperparameters and gives less transparent reasons.

Cold start and sparsity. A new user with zero ratings has no neighbours and no rated items, so neither method can produce anything. A new movie with zero ratings has no similarity to any other movie and never appears in a list. Few co-ratings produce extreme similarities, which is why the confidence filter is needed.

What worked.

- Looking at the raw evidence (co-rated counts and total raters) exposed a problem that the RMSE of the item-based model barely showed.
- Recomputing the key numbers in a separate script confirmed every value, and printing the neighbours' co-rated counts there is how I noticed the problem.

What surprised me or did not work.

- Item-based predictions stay close to the user's own average (user 1: mean 3.610, Star Wars predicted 3.698), so small differences decide the ranking and niche titles still come first. This helps catalog discovery but gives less familiar recommendations.
- Weighting changed the item-based RMSE only slightly (1.0335 → 1.0281); for the item-based lists, the real fix was counting distinct users in the confidence filter.

Next improvement. Compare Pearson correlation, add popularity-aware ranking for item-based lists, and use matrix factorization for the prediction panel.

6. AI Usage Disclosure

AI tools used. Pi Coding Agent 0.85.1; both implementation sessions ran on the GLM-5.3 Flash model through the Z.ai provider.

How I used AI. I planned the prompts myself and gave two separate implementation tasks to a coding agent: the first build, then an improvement round after my own testing found the evidence problems. I also used it to draft the report text.

What I verified myself. I recomputed the co-rated cosine by hand, reran the test and analysis, recomputed the Top-5 lists and pair predictions independently in Python, checked the paper metadata through Crossref, and opened every reference. Following the course material [8], I used AI to speed up the implementation, not to replace the diagnosis step.

What I trusted without verification. I did not reimplement matrix factorization independently; its numbers come from the analysis script.

Session log. The attached `argrishkina_a03_session.json` is a privacy-redacted audit generated from both task-only implementation sessions. It contains tool calls, file changes, and milestones, but excludes conversational content and replaces local account identifiers. It is not an unmodified Pi export. The prompts and notes are in `week3/prompt.md`.

References

[1] J. S. Jin, "Collaborative Filtering Movie Recommender (week3)," *GitHub*, 2026. [Online]. Available: https://github.com/dryjins/RecSys-LLMs/tree/main/week3. [Accessed: 2026-09-30].

[2] Google for Developers, "Collaborative filtering," *Machine Learning: Recommendation Systems*. [Online]. Available: https://developers.google.com/machine-learning/recommendation/collaborative/basics. [Accessed: 2026-09-30].

[3] Google for Developers, "Matrix factorization," *Machine Learning: Recommendation Systems*. [Online]. Available: https://developers.google.com/machine-learning/recommendation/collaborative/matrix. [Accessed: 2026-09-30].

[4] GroupLens Research, "MovieLens 100K Dataset." [Online]. Available: https://grouplens.org/datasets/movielens/100k/. [Accessed: 2026-09-30].

[5] GroupLens Research, "ml-100k README." [Online]. Available: https://files.grouplens.org/datasets/movielens/ml-100k-README.txt. [Accessed: 2026-09-30].

[6] J. L. Herlocker, J. A. Konstan, A. Borchers, and J. Riedl, "An algorithmic framework for performing collaborative filtering," in *Proc. 22nd Annu. Int. ACM SIGIR Conf.*, 1999, pp. 230–237, doi: 10.1145/312624.312682.

[7] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-based collaborative filtering recommendation algorithms," in *Proc. 10th Int. Conf. World Wide Web*, 2001, pp. 285–295, doi: 10.1145/371920.372071.

[8] MIT Ad Hoc Committee on AI Use in Teaching, Learning, and Research Training, “Report of MIT’s Ad Hoc Committee on AI Use,” Massachusetts Institute of Technology, Aug. 13, 2026, Sec. 2.7 “Augmentation not automation.”